



Penetration Testing

Arcane

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	6
Security Rating	7
Severity Definitions	8
Status Definitions	9
Penetration Test Findings	10
Conclusion	34
Our Methodology	35
Disclaimers	37
About Hashlock	38

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Arcane team partnered with Hashlock to conduct a security Penetration Test of their Arcane Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Arcane is a Web3 privacy-focused infrastructure project building confidential and compliance-oriented blockchain solutions for decentralized finance and cross-chain applications. The protocol focuses on enabling private transactions, shielded asset transfers, and privacy-preserving DeFi interactions through advanced cryptographic mechanisms and decentralized infrastructure. Arcane's ecosystem includes components such as private liquidity infrastructure, privacy-enhanced wallets, cross-chain interoperability mechanisms, and institutional-grade compliance tooling designed to support secure and confidential on-chain activity across multiple blockchain ecosystems.

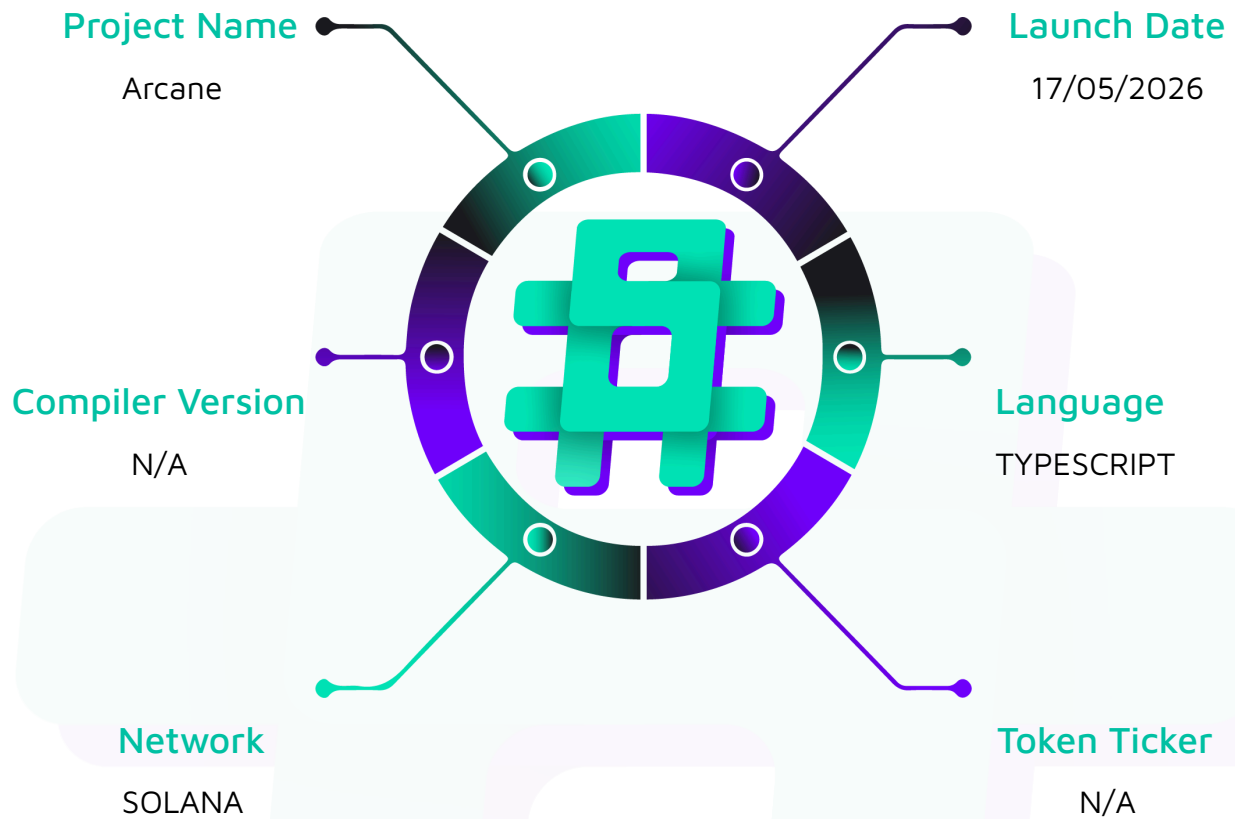
Project Name: Arcane

Project Type: DeFi

Website: <https://arcaneprivacy.com/>

Logo:



Visualised Context:

Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Arcane project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Arcane Code Base
Platform	Web apps & APIs
Penetration Test Date	April, 2026
Repository (1)	https://github.com/ArcanePrivacy/Relayer
Repository (2)	https://github.com/ArcanePrivacy/Front-end
Repository (3)	https://github.com/ArcanePrivacy/Indexer
GitHub Commit Hash (1)	87bb7d5c21081ba8f77e49e6df9c60d320b40581
GitHub Commit Hash (2)	b1391e010fed54dc9a62ad6c72abcaf5ef2b2788
GitHub Commit Hash (3)	854f4a550ae83c2bd34e95ee4ccc06594fdd328c
Fix Review GitHub Commit Hash	

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High-severity vulnerabilities

8 Medium-severity vulnerabilities

8 Low-severity vulnerabilities

Caution: Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] FrontEnd - ArcanePayPage#handleFileChange - CSV-imported private keys held in React state

Description

The Arcane Pay UI parses CSV-uploaded private keys via FileReader, stores raw key strings on each entry inside React state for the lifetime of the tab, and only obfuscates them at the display layer. Any browser extension, devtools snapshot, or in-memory dump may recover them.

Vulnerability Details

src/components/pages/ArcanePayPage.jsx:45-55 parseCSVKeys reads the CSV and calls setSourceWallets(parsed.map(k => ({ key: k, ... }))); The display layer masks via maskKey() (:40-43) but the underlying state remains plaintext. This means that those keys are persistent in browser memory.

```
reader.onload = (ev) => {  
  
  const parsed = parseCSVKeys(ev.target.result);  
  
  setSourceWallets(parsed.map(k => ({ key: k, ... })));  
  
};
```

Impact

The private keys are at risk of compromise if e.g. a malicious extension is loaded and they are easy to recover once access to browser memory is obtained in a similar way.

Recommendation

Never persist raw private keys in React state. Clear the memory immediately after processing the keys.

Status

Resolved



[H-02] Indexer - relayer-signup-sync.js #refreshRelayerHttpStatuses - SSRF via relayer URL

Description

Anyone who registers a relayer on the on-chain registry program controls a `url` string that the indexer fetches every cron tick with `redirect: 'follow'`. The same fetch sink consumes URLs that flowed through SNS resolution, DNS TXT records and the `*.sol-domain.org` gateway redirect.

Vulnerability Details

`refreshRelayerHttpStatuses` (`relayer-signup-sync.js:758-832`) loads every `RelayerUrl` document with `registeredOnChain !== false`, builds `statusUrl = ${doc.url}${path}` from the on-chain string, and issues `fetch(statusUrl, { redirect: 'follow' })` from the indexer host. `normalizeRelayerUrl` (`relayer-signup-sync.js:226-249`) accepts any well-formed `http(s)` URL, including e.g. `http://127.0.0.1:6379`, `http://169.254.169.254/`, `http://10.0.0.0/8` and IPv6 link-local.

```
// relayer-signup-sync.js:778-790
const statusUrl = `${base}${path}`;
...
const res = await fetch(statusUrl, {
  method: 'GET',
  signal: ac.signal,
  redirect: 'follow',
  headers: { Accept: 'application/json, text/plain, */*' },
});
```

Impact

Blind SSRF from the indexer host to any internal service reachable on its network: cloud metadata services, MongoDB on localhost:27017, internal admin panels, Redis, etc. A request to such services may lead to DoS or executing specific actions on the applications side.

Recommendation

Resolve every fetch target via dns.lookup first and reject loopback, link-local (169.254.0.0/16), private networks, or enforce specific format / whitelist URLs manually. Set redirect: 'manual' and revalidate the resolved IP after each 3xx status.

Status

Resolved

Medium

[M-01] Relay - Worker/RPC integration - Missing RPC timeouts

Description

The relay performs multiple external RPC calls during validation, fee estimation, submission, and confirmation, but no request deadline or circuit breaker is applied. A slow or hanging RPC provider can therefore block the single worker indefinitely.

Vulnerability Details

The connection object is created with only a commitment setting. The worker then awaits `getAccountInfo`, `networkState.fetch`, `getLatestBlockhash`, `simulateTransaction`, `getFeeForMessage`, `confirmTransaction`, and `getTransaction` directly. Because there is no timeout wrapper, retry budget, or fallback provider, a degraded RPC can stall queue consumption and create a cascading denial of service condition.

```
// src/modules/connection.js

const getConnection = () => {

  if (!rpcUrl) {

    throw new Error('RPC_URL is not set. Please set it in your .env file')

  }

  const commitment = 'confirmed'

  const config = {

    commitment,

  }

  if (rpcUrl.startsWith('wss://')) {

    config.wsEndpoint = rpcUrl

  }

}
```

```

    }

    return new web3.Connection(rpcUrl.replace('wss://', 'https://'), config)
  }

  // src/worker.js

  const accountInfo = await connection.getAccountInfo(publicKey)

  ...

  const networkState = await program.account.networkState.fetch(networkStatePDA)

  ...

  tx.recentBlockhash = (await
  provider.connection.getLatestBlockhash('confirmed')).blockhash

  ...

  const simulationResult = await provider.connection.simulateTransaction(tx, [keypair])

  ...

  const feeResponse = await provider.connection.getFeeForMessage(message, 'confirmed')

  ...

  const confirmation = await program.provider.connection.confirmTransaction(

```

Impact

RPC degradation can stop withdrawal processing even when the API and Redis remain healthy.

Recommendation

Wrap each RPC stage in an explicit timeout, define bounded retries with backoff, and fail jobs fast when deadlines are exceeded. A secondary RPC endpoint would improve resilience.

Status

Acknowledged



[M-02] Relayer - Network setup - Redis is reachable without authentication from the host and all containers on the shared bridge

Description

Redis is bound to the host loopback and also attached to the shared proxy-network Docker bridge. Because no password, ACL, or TLS is configured, any local process on the host and any container on that bridge can read and modify the relayer's queue and operational state.

Vulnerability Details

The compose file publishes Redis on 127.0.0.1:6379 and connects redis, solana-server, solana-healthWatcher, solana-worker1, nginx-proxy, and acme-companion to the same bridge network. The application uses plain redis:// connections with no authentication. That means Redis is not internet-exposed by this file alone, but it is reachable as a second-order target from any compromise of a peer container or any service with loopback access on the host. I did not identify an SSRF-style path in this repository that lets a public HTTP caller directly pivot into Redis.

```
# docker-compose.yml

services:

  redis:

    image: redis

    restart: always

    command: [redis-server, --appendonly, 'yes']

    ports:

      - '127.0.0.1:6379:6379'

    networks:

      - proxy-network

...

nginx-proxy:
```

```
networks:

  - proxy-network

...

acme-companion:

  networks:

    - proxy-network
```

Impact

Any attacker that gains code execution in a peer container or a foothold on the host can tamper with queued jobs, read job metadata, erase health/error evidence, or disrupt processing. Redis-specific post-exploitation primitives such as arbitrary file writes become relevant once that access exists.

Recommendation

Remove the host port binding unless it is operationally required, place Redis on a dedicated internal-only Docker network, and require authentication at a minimum.

Status

Resolved

[M-03] Relay - server.js / router.js - No application-level rate limiting**Description**

The Express server has no rate-limiting middleware. The /v1/arcaneWithdraw and /relay endpoints accept POST requests that trigger RPC calls, transaction simulation, and on-chain submission. Each request consumes significant compute and RPC quota.

Impact

An attacker can flood the relay with withdrawal requests, exhausting RPC rate limits, filling the Redis queue, and causing legitimate transactions to be delayed or fail. The relay operator bears the cost of all RPC calls and transaction fees for simulated transactions.

Recommendation

Add express-rate-limit middleware to the Express application, especially on the withdrawal endpoints.

Status

Acknowledged

[M-04] Relayer - Public relay API#arcaneWithdraw - Unlimited job submission enables persistent denial of service

Description

The public write endpoints accept any request that matches the JSON schema and immediately enqueue it for processing. Because jobs are retained and no rate limiting, authentication, or queue cap is enforced, a remote attacker can continuously create work and exhaust Redis storage, worker capacity, and RPC quota.

Vulnerability Details

Both /v1/arcaneWithdraw and /relay are exposed without authentication. The controller only performs format validation, then persists the request into Bull and stores a secondary Redis mapping. Completed jobs are not removed (the removeOnComplete: true is commented out), failed-job retention is not bounded, and the job:<uuid> mapping never expires.

```
// src/queue.js

async function postJob({ type, request }) {

  const id = uuid()

  const job = await queue.add(

    {

      id,

      type,

      status: status.QUEUED,

      ...request, // proof, args, ?contract

    },

    {

      //removeOnComplete: true

    },

  ),
```

```
)  
  
await redis.set(`job:${id}`, job.id)  
  
return id  
}
```

Impact

An unauthenticated attacker can fill Redis with persistent jobs, keep the relayer busy simulating and confirming invalid withdrawals, and degrade or halt service for legitimate users.

Recommendation

Add IP and wallet-level rate limiting and remove or prune completed and failed jobs.

Status

Acknowledged

[M-05] Frontend - OnboardingFlow#useEffect & zkProof#loadScript - Third-party scripts loaded without SRI

Description

Two separate code paths inject third-party JavaScript into the page without a Subresource Integrity (integrity) attribute: the Unicorn Studio CDN in onboarding, and snarkjs.min.js (both statically in index.html and dynamically via getSnarkjs). A compromise of cdn.jsdelivr.net may lead to unknowingly serving arbitrary Javascript.

Vulnerability Details

```
// src/components/pages/OnboardingFlow.jsx:95-110
useEffect(() => {

    const SDK_URL =
'https://cdn.jsdelivr.net/gh/hiunicornstudio/unicornstudio.js@v2.0.5/dist/unicornStudio.umd.js';

    ...

    const script = document.createElement('script');

    script.src = SDK_URL;

    script.onload = initUS;

    document.head.appendChild(script);

// index.html:13 and src/lib/zkProof.js:244-262:
<script src="%BASE_URL%assets/snarkjs.min.js"></script>
[...]
```

```
function loadScript(src) {

    ...

    const s = document.createElement('script');

    s.src = src;

    s.async = true;

    s.onload = () => resolve();
```

```
s.onerror = () => reject(new Error(`Failed to load ${src}`));

document.head.appendChild(s);
}
```

Impact

If the scripts are compromised, they will be still loaded as legitimate part of web application, potentially serving malicious content (e.g. fraudulent tx signing).

Recommendation

Implement SRI
(https://developer.mozilla.org/en-US/docs/Web/Security/Practical_implementation_guides/SRI) to those scripts.

Status

Resolved

[M-06] Frontend - App - Missing security headers

Description

The app has no setup for security headers e.g. CSP, there is no client-side mitigation against a single XSS or supply-chain compromise.

Impact

Any same-origin script execution can fetch any origin, read all localStorage deposit secrets.

Recommendation

Ship a strict Content-Security-Policy (the protocol speaks to a small known set of origins: Solana RPC, Arcane indexer/risk, Helius, CoinGecko, jsDelivr) plus Referrer-Policy: no-referrer, X-Frame-Options: DENY, and SRI on every external script.

Status

Resolved

[M-07] Frontend - program.js#RANGE_API_KEY - Hardcoded API key

Description

RANGE_API_KEY is exported with a hard-coded fallback bearer token. Vite embeds the literal in every public build because .env.example does not declare VITE_RANGE_API_KEY. The token is still imported by the withdraw components even though all call sites are commented out.

Vulnerability Details

```
export const RANGE_API_KEY = import.meta.env.VITE_RANGE_API_KEY  
  
  || 'cmn8n4p7i0002pc0108xnhx9g.6h9P1nb7wryD9SN78HpdYfHqhBYhT7F2';
```

Impact

Anyone who downloads the bundle can call api.range.org under the protocol's account until the token is rotated, exhausting the paid quota and exposing the project to abuse-cost claims from the upstream provider.

Recommendation

Implement SRI: Delete the fallback string, remove the unused import.

Status

Resolved

Low

[L-01] Relayer - worker.js / connection.js - Relayer acts on non-finalized blockchain state

Description

The entire relayer uses a 'confirmed' commitment level for all Solana RPC interactions. The worker marks a job as CONFIRMED after receiving a confirmed commitment response, which does not guarantee finality on Solana.

```
// src/modules/connection.js:9

const commitment = 'confirmed'

// src/worker.js:178

const confirmation = await program.provider.connection.confirmTransaction(
  { signature, blockhash, lastValidBlockHeight },
  'confirmed',
)
```

On Solana, confirmed means optimistic confirmation (~400ms) by a supermajority of validators. While Solana reorgs are rare, they are not impossible during network instability. As per Solana docs, "Using the default commitment level finalized will eliminate any risk that the blockhash you choose will belong to a dropped fork. The tradeoff is that there is typically at least a 32-slot difference between the most recent confirmed block and the most recent finalized block. This tradeoff is pretty severe and



effectively reduces the expiration of your transactions by about 13 seconds, but this could be even more during unstable cluster conditions.” (<https://solana.com/developers/guides/advanced/confirmation>)

In a theoretical reorg scenario, a withdrawal transaction marked as CONFIRMED by the relayer could be dropped. The nullifier would not be recorded on-chain, potentially allowing double-withdrawal.

Recommendation

For the final status update, use the 'finalized' commitment level.

Status

Resolved

[L-02] Relayer - Dockerfile - Container runs as root user

Description

The Dockerfile does not specify a non-root USER directive, causing the application to run as root inside the container.

```
FROM node:20

WORKDIR /app

COPY package.json yarn.lock ./

RUN yarn && yarn cache clean --force

COPY . .

EXPOSE 8000

ENTRYPOINT ["yarn"]

# No USER directive
```

If an attacker achieves code execution within the container, they gain root access, allowing for full takeover and privilege escalation, given that the Docker socket is mounted.

Recommendation

Add a non-root user: RUN addgroup --system app && adduser --system --ingroup app app followed by USER app.

Status

Resolved

[L-03] Relayer - router.js - Missing security response headers

Description

The router sets X-Frame-Options: DENY, but is missing Content-Security-Policy, Strict-Transport-Security, and X-Content-Type-Options.

Without X-Content-Type-Options: nosniff, browsers may MIME-sniff JSON responses as HTML. Missing HSTS allows SSL stripping attacks. The index route returns raw HTML without CSP.

Recommendation

Use the helmet middleware: `app.use(helmet())`

Status

Resolved

[L-04] Relayer - server.js - No explicit request body size limit on JSON parser

Description

The Express JSON parser uses the default 100KB limit, which is larger than necessary for withdrawal requests (~1KB). Combined with absent rate limiting, this increases the surface for memory/bandwidth exhaustion.

Recommendation

Set an explicit limit: `app.use(express.json({ limit: '5kb' }))`

Status

Resolved

[L-05] FrontEnd - arcaneProgram#getEncryptionKey - predictable signing message enables cross-site replay of the note-decryption key

Description

The 32-byte note-encryption key is `nacl.hash(wallet.signMessage('arcane:notes:v1')).slice(0,32)`. The constant input has no domain/origin/nonce binding, so any other site can prompt the same wallet for the same string and recover the identical key, then read the encrypted notes blob from the public notes PDA. However, the function seems to be unused.

Vulnerability Details

`src/lib/arcaneProgram.js:231-241` defines `getEncryptionKey`.

```
const message = new TextEncoder().encode('arcane:notes:v1');  
  
const signature = await wallet.signMessage(message);  
  
const key = nacl.hash(signature).slice(0, 32);
```

Impact

If the function is used later, a phishing site that gets the wallet to sign `arcane:notes:v1` once recovers every backed-up note for that wallet, decrypts every deposit's nullifier/secret, and produces valid withdraw proofs draining each commitment.

Recommendation

Bind the signing message to a domain, wallet pubkey, and on-chain-stored salt: e.g. `Arcane Notes Encryption v2\nDomain: arcaneprivacy.com\nWallet: <pubkey>\nNonce: <random>`.

Status

Acknowledged

[L-06] Indexer - server.js#syncSolanaTransactions - indexer credits Deposit/Withdraw events at confirmed status with no reorg handling

Description

The indexer fetches and persists Deposit and Withdraw events at commitment: 'confirmed' and never re-checks them at finalized.

Vulnerability Details

`new Connection(SOLANA_RPC_URL, 'confirmed')` (server.js:30) is the global connection used for every `getSignaturesForAddress` and `getTransaction` call. There is no second pass at finalized. A reorg-dropped Deposit therefore stays in MongoDB and continues to be served by `/commitments`, `/tx-status`, `/validate-nullifier` and `/wallet-balance`.

```
// server.js:30
const connection = new Connection(SOLANA_RPC_URL, 'confirmed');

// server.js:262-263
{ before, limit: 1000, commitment: 'confirmed' }

// server.js:290-293
const tx = await getTransactionPaced(connection, signature, {
  commitment: 'confirmed',
  maxSupportedTransactionVersion: 0,
});
```

Impact

Stale rows after a reorg cause the dApp to build Merkle proofs against commitments the on-chain pool has dropped (legitimate withdrawals fail), or, on the nullifier path, to mark a hash as "spent" when the canonical chain never spent it (legitimate withdrawal blocked).

Recommendation

Read events at confirmed for liveness but only persist them after a finalized re-fetch.

Status

Acknowledged



[L-07] Indexer - range-risk-server.js#address-risk - verbose error messages

Description

The catch handler returns the raw `err.message` from a Node `fetch` failure, exposing DNS errors, TLS chain mismatches, internal IP/port hints, and proxy hops.

Vulnerability Details

`range-risk-server.js:91-97` returns `details: err.message || String(err)` in the JSON 502 body. `node-fetch/undici` error messages can include resolved IP addresses, port numbers, TLS chain identifiers and proxy hostnames.

```
// range-risk-server.js:92-97
console.error('Range API request failed:', err.message || err);
return res.status(502).json({
  error: 'Failed to reach Range API',
  details: err.message || String(err),
});
```

Impact

Information disclosure that aids reconnaissance: the structure of the deployment topology and technical stack becomes visible to any anonymous caller.

Recommendation

Return generic `details: 'upstream unreachable'` or omit the field entirely. Keep the verbose `console.error` for operators only.

Status

Acknowledged

[L-08] Indexer - `server.js#commitments` - returns HTTP 500 on missing input instead of 400

Description

The `/commitments` handler returns a 5xx status code for what is a client error (missing query parameter), causing the server to respond with a 500 error, typically indicating a backend failure.

Vulnerability Details

`server.js:655-657` returns `res.status(500).json({ error: "Denomination parameter is required" })` when `denomination` is absent, while the sibling endpoint at `server.js:720-728` (`/deposit-count-by-commitment`) correctly uses 400 for the same kind of missing-parameter case.

```
// server.js:655-657
if (!denomination) {
  return res.status(500).json({ error: "Denomination parameter is required" });
}
```

Impact

5xx false alarms in monitoring, possibly misleading responses that suggest a server bug instead of a missing parameter.

Recommendation

Change the status code to 400.

Status

Acknowledged

Conclusion

After Hashlock's analysis, the Arcane project seems to have a sound and well-tested code base; now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

